Appendices

Appendix: Pipeline Stage Reference

Preparation and analysis I

Table 1: Single-project DAG reference from default `pipeline.yaml` (names shown in dependency order). Scripts live under `scripts/`.

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Clean Output Directories	`_run_clean_outputs`	prior `projects/<name>/output/`, mirrored `output/<name>/` targets	emptied trees	Blocking
Environment Setup	`scripts/pipeline/stage_00_setup.py`	toolchain probes	scaffold dirs, env exports	Blocking
Infrastructure Tests	`scripts/pipeline/stage_01_test.py --infra-only --infra-scope pipeline-smoke`	`tests/infra_tests/`	smoke-test results	Required smoke tests must pass
Project Tests	`scripts/pipeline/stage_01_test.py --project-only`	`projects/<name>/tests/`	coverage artefacts	Nonempty tests, own coverage, verifier, and runner
Project Analysis	`scripts/pipeline/stage_02_analysis.py`	thin scripts	`figures/`, `data/`, reports	Blocking

Search, provenance and rendering I

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Connector Search	`scripts/pipeline/stage_08_connector_search.py`	`manuscript/config.yaml`	`output/data/connector_search/`	opt-in (`science` tag); fresh policy-bound skip if unconfigured
Provenance Record	`scripts/pipeline/stage_09_provenance_record.py --stage "Connector Search"`	prior stage outputs	`.provenance/graph.json`	opt-in (`provenance` tag); fresh policy-bound skip if unconfigured
PDF Rendering	`scripts/pipeline/stage_03_render.py`	`manuscript/`, placeholders	`.pdf`/`.tex` bundles	Blocking
Output Validation	`scripts/pipeline/stage_04_validate.py`	render tree	Markdown + PDF diagnostics JSON	Enabled formats, bookends, and provenance block; prose diagnostics advisory

Review and delivery I

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
LLM Manuscript Review	`scripts/pipeline/stage_06_llm_review.py --reviews-only`	resolved manuscript artefacts	textual reviews	Fresh policy-bound skip; required live acceptance remains blocking
LLM Translations	`scripts/pipeline/stage_06_llm_review.py --translations-only`	abstract metadata	multilingual snippets	Fresh policy-bound skip; required live acceptance remains blocking
Copy Outputs	`scripts/pipeline/stage_05_copy.py`	validated tree	mirrored `output/<name>/…`	Blocking

Optional packaging and publication I

Stage name	Script / method	Primary inputs	Outputs / artefacts	Failure mode
Ebook Generation	`scripts/pipeline/stage_11_ebook.py`	rendered combined markdown	`output/ebook/` (EPUB/MOBI/DOCX)	opt-in (`ebook` tag); blocking when invoked
docxplus Export	`scripts/pipeline/stage_13_docxplus.py`	project source tree + metadata	`output/docxplus/` (.docx, .docxplus)	opt-in (`docxplus` tag); blocking when invoked
Metadata Package	`scripts/pipeline/stage_12_metadata.py`	`manuscript/config.yaml`	`output/metadata/` (ONIX/JSON/OPF)	opt-in (`metadata` tag); blocking when invoked
Executable Bundle	`scripts/runner/bundle_executable.py`	project tree + outputs	container bundle manifest	opt-in (`bundle` tag)
Archival Publication	`scripts/runner/archive_publication.py`	bundle + deliverables	archival deposit manifest	opt-in (`archival` tag)

Optional packaging and publication I

`scripts/pipeline/stage_07_executive_report.py` provides separate **multi-project executive reporting**, not an additional single-project DAG node. `execute_multi_project.py` writes aggregate summaries after project execution; this does not imply automatic invocation of that separate Stage 07 script.

Opt-in stages also prevent completion when invoked and unsuccessful. An allowed skip requires a fresh policy-bound reason receipt; it is not a successful execution. The executable DAG and its contracts remain authoritative.